



LeetCode 1051: Height Checker

1. Problem Title & Link

Title: LeetCode 1051: Height Checker

Link: <https://leetcode.com/problems/height-checker/>

2. Problem Statement (Short Summary)

A group of students is standing in a line.

You are given an array:

heights

representing the current order of students.

Students should stand in:

Non-decreasing order

(sorted ascending order).

Return the number of indices where:

Current height

≠

Expected height

Example

Input:

heights = [1,1,4,2,1,3]

Sorted order:

[1,1,1,2,3,4]

Compare:

Current : [1,1,4,2,1,3]

Expected: [1,1,1,2,3,4]

Different positions:

Index 2

Index 4

Index 5

Answer:

3

3. Examples (Input → Output)

Example 1

Input:

[1,1,4,2,1,3]



Output:

3

Example 2

Input:

[5,1,2,3,4]

Output:

5

Example 3

Input:

[1,2,3,4,5]

Output:

0

Already sorted.

Example 4

Input:

[2,1]

Output:

2

4. Constraints

$1 \leq \text{heights.length} \leq 100$

$1 \leq \text{heights}[i] \leq 100$

Important:

Need mismatch count

Not the number of swaps.

5. Core Concept (Pattern / Topic)

Sorting

Secondary Topics:

- Array Comparison
- Counting



6. Thought Process (Step-by-Step Explanation)

Understanding the Problem

We have:

Current arrangement

and

Expected arrangement

Expected arrangement is simply:

Sorted array

Example

Current:

[1,1,4,2,1,3]

Expected:

[1,1,1,2,3,4]

Compare index by index:

Index	Current	Expected	Match ?
0	1	1	✓
1	1	1	✓
2	4	1	✗
3	2	2	✓
4	1	3	✗
5	3	4	✗

Total mismatches:

3

Key Observation

The expected arrangement is simply:

sorted(heights)

Then compare both arrays.

Key Insight

We don't need to simulate swaps.

We only need:

Count positions where values differ.



7. Visual / Intuition Diagram (ASCII Diagram)

```
Current:
1 1 4 2 1 3
Sorted:
1 1 1 2 3 4
Comparison:
1 1 4 2 1 3
1 1 1 2 3 4
    ↑  ↑  ↑
Mismatch count:
3
```

8. Pseudocode (Language Independent)

```
expected = sorted copy

count = 0

for each index
    if heights[i] != expected[i]
        count++

return count
```

9. Code Implementation

Python

```
class Solution:
    def heightChecker(self, heights):

        expected = sorted(heights)

        count = 0

        for i in range(len(heights)):

            if heights[i] != expected[i]:
                count += 1

        return count
```

Java

```
import java.util.Arrays;

class Solution {
```



```
public int heightChecker(  
    int[] heights) {  
  
    int[] expected =  
        heights.clone();  
  
    Arrays.sort(expected);  
  
    int count = 0;  
  
    for(int i = 0;  
        i < heights.length;  
        i++) {  
  
        if(heights[i] != expected[i])  
            count++;  
    }  
  
    return count;  
}
```

10. Time & Space Complexity

Time Complexity

Sorting:

$O(n \log n)$

Comparison:

$O(n)$

Overall:

$O(n \log n)$

Space Complexity

$O(n)$

For sorted copy.

11. Common Mistakes / Edge Cases

Mistake 1

Counting swaps.

Question asks:

Mismatch count

not:

Minimum swaps

**Mistake 2**

Sorting original array.

Then comparison becomes impossible.

Need a copy.

Mistake 3

Comparing values instead of positions.

Position matters.

Edge Cases**Already Sorted**

Input:

[1,2,3,4]

Output:

0

Reverse Sorted

Input:

[4,3,2,1]

Sorted:

[1,2,3,4]

Every position differs.

Output:

4

Single Student

Input:

[5]

Output:

0

12. Detailed Dry Run

Input:

[1,1,4,2,1,3]

Expected:

[1,1,1,2,3,4]



0	1	1	0
1	1	1	0
2	4	1	1
3	2	2	1
4	1	3	2
5	3	4	3

Return:

3

13. Common Use Cases (Real-Life / Interview)

Ranking Systems

Compare current rank vs expected rank.

Classroom Arrangement

Check students out of place.

Data Validation

Compare actual order vs sorted order.

Interview Purpose

Tests:

Sorting

Array Comparison

Counting

14. Common Traps (Important!)

Trap 1

Thinking sorting itself is the answer.

Need mismatch count after sorting.

Trap 2

Trying to calculate minimum swaps.

Not required.

Trap 3

Sorting the original array.

Loses the original ordering information.



15. Builds To (Related LeetCode Problems)

LC 1051 - Height Checker



LC 75 - Sort Colors



LC 912 - Sort an Array



LC 977 - Squares of Sorted Array



LC 56 - Merge Intervals

These improve sorting intuition.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Sort + Compare	$O(n \log n)$	$O(n)$	Best
Counting Sort	$O(n)$	$O(100)$	Advanced

Approach 1: Sort + Compare (Chosen)

Most intuitive.

Most interview-friendly.

Approach 2: Counting Sort

Because:

$1 \leq \text{height} \leq 100$

We can use frequency counting.

Achieves:

$O(n)$

time.

Advanced optimization.

17. Why This Solution Works (Short Intuition)

The sorted version represents the expected arrangement.

Every index where the original array differs from the sorted array corresponds to a student standing in the wrong position.



Counting these mismatches gives the answer.

18. Variations / Follow-Up Questions

Follow-Up 1

Return the mismatched indices.

Example:

[2,4,5]

Follow-Up 2

Return students already in correct positions.

Follow-Up 3

Calculate minimum swaps needed.

Different problem.

Follow-Up 4

Implement using Counting Sort.

Expected interview follow-up because heights are limited to:

1..100

Interview Takeaway

The key pattern is:

Sort

+

Compare

Whenever you hear:

Expected Order

Correct Arrangement

Sorted Position

think:

Create Sorted Copy

Compare Index by Index

This is a common pattern in easy-to-medium sorting and array comparison problems.